Subscription Churn Prediction — Mini Production ML System

Design Document | Student ID: 2025em1100213 | Production ML Systems | August 2026

Code repository: https://github.com/2025em1100213-lgtm/SemesterAssignment/

1. Problem Definition and Metrics

Subscription businesses lose recurring revenue whenever a customer cancels, and by the time a cancellation actually happens it is usually too late to intervene. This system predicts, for each active customer, the probability that they will churn in the near term — a binary classification task using contract, billing, and subscribed-services fields, with target label churn ∈ {0, 1}. The prediction is intended to feed a real retention workflow, where a support or retention agent decides whether to offer an incentive or place an outreach call, which forces concrete production decisions about latency, feature staleness, and monitoring rather than being a passive offline report.

Metric	Why it matters here
ROC AUC (primary)	Threshold-independent ranking; retention teams work down a ranked list, and churn is imbalanced (~26.5% positive).
Accuracy (secondary)	Sanity check only — predicting "no churn" always already scores ~73.5%.
Precision@k (future)	Matches how a budget-limited retention team would actually consume the ranked list.

2. Data and Feature Design

Source: the public IBM/Kaggle Telco Customer Churn dataset — 7,043 real residential customers, one row per customer, covering contract type, billing, tenure, and subscribed services. Target label: Churn (Yes/No), mapped to churn ∈ {0,1}; churn rate across the dataset is 26.5%, a realistic, moderately imbalanced split typical of subscription businesses. Cleaning and assumptions: TotalCharges ships as a string with 11 blank values, all belonging to brand-new customers with tenure = 0 who have not been billed yet, so these are coerced to numeric and filled with 0.0; Yes/No and three-way ("no internet service" / "no phone service") columns are collapsed to clean 0/1 flags; Contract, PaymentMethod, and InternetService are mapped to a fixed, lowercase vocabulary so the encoder never has to special-case raw label text at serving time.

Engineered features (5+ required)

#	Feature	Type	Description
1	charges_per_tenure_month	Online / ratio	Total spend normalized by tenure length
2	num_addon_services	Online / aggregation	Count of subscribed add-ons
3	is_high_value	Online / threshold	Monthly spend above $70
4	tenure_bucket	Online / binning	New / established / loyal tenure segments

#	Feature	Type	Description
5	is_new_high_risk	Online / interaction	Month-to-month contract AND tenure ≤ 12 mo
6-8	contract, payment, internet type	Online / encoding	Fixed-vocabulary categorical encodings

Offline vs. online, and training–serving skew

All sixteen model features are cheap, stateless functions of fields already present on a single customer record, so every one of them is computed online, per request, at serving time; a genuine rolling-window feature such as a 90-day support-ticket trend would instead need to move to an offline feature table with an online lookup, since it cannot be derived from a single request payload. The most safety-critical design decision in the project is training–serving skew mitigation: a single shared FeatureEngineer class with a stateless transform method is imported unchanged by both the offline training pipeline and the online FastAPI serving layer, so there is no second implementation anywhere in the codebase. A regression test asserts that transforming one row through the single-record path produces bit-identical output to transforming it as part of a batch, so serving-time feature values can never silently diverge from what the model was trained on.

Data ingestion pipeline

Because the Telco export is a single static file rather than a dated warehouse table, it is deterministically split into 10 simulated daily CSV files to mimic a real "one file lands per day" feed. The ingestion step then performs the actual production pattern required: it scans the incoming folder for new files, validates schema (required columns present, files with a null rate above 50% are rejected), appends and deduplicates rows into an append-only training table, and logs the row count and date for each file it processes. Processed files are moved to an _ingested/ folder afterward so re-running the job never double-counts. This run ingested all 7,043 rows across 10 files with zero validation failures.

3. Model Choice and Evaluation

The training pipeline is fully repeatable: load the processed table, take a stratified 70/15/15 train/val/test split, fit a StandardScaler on the training split only, train both a baseline and a candidate model, evaluate both on the validation and held-out test sets, apply the promotion guardrail, and save all artifacts. The baseline is logistic regression, chosen because it is simple, fast to train, and interpretable — its coefficients double as feature importances and it sets a reasonable lower bound. The candidate is gradient boosting, chosen because it can capture non-linear feature interactions a linear model cannot, such as a month-to-month contract combined with a low add-on count being riskier than either factor alone. The promotion guardrail requires the candidate to clear an absolute quality bar (validation AUC ≥ 0.80) and to not regress meaningfully against the baseline (no more than 0.01 worse).

Model	Val AUC	Val Acc	Test AUC	Test Acc
Baseline — Logistic Regression	0.8532	0.8030	0.8148	0.7796
Candidate — Gradient Boosting	0.8507	0.7992	0.8248	0.7843

Decision: candidate promoted — 0.8507 ≥ 0.80 and the 0.0025-point gap to baseline is within tolerance; its test AUC (0.8248) is actually higher than baseline's (0.8148). Artifacts (models, scaler, registry.json, JSON/Markdown eval reports) are saved to models/ and artifacts/eval/.

4. Serving and Inference Pattern

Pattern chosen: online request-response

Using the standard framing: a human (a support or retention agent) is waiting on the score; the latency budget is sub-100 ms since the call gates a UI interaction; and the use case is naturally per-event, triggered whenever a customer contacts support, rather than something that fits a fixed nightly batch schedule. A pure batch, score-everyone-nightly pattern was rejected because it would leave scores up to 24 hours stale exactly when they matter most, and a hybrid precompute-plus-lookup pattern was rejected as unnecessary complexity given how cheap every feature here is to compute inline.

API design and performance

A FastAPI service exposes POST /predict, which returns churn_probability, churn_prediction, and model_version, plus GET /health for readiness probes. A 200-request load test against a running instance measured mean latency 21.8 ms, p95 34.8 ms, p99 38.37 ms, zero errors, and throughput of 45.88 requests/second — comfortably inside the latency budget.

Containerization

The Dockerfile uses a python:3.11-slim base image with dependencies installed from requirements.txt before any application code is copied in, so the dependency layer is cached and unaffected by day-to-day code changes. Only what the serving path actually needs is copied into the image — src/, serving/, configs/, and models/ — keeping the image lean; port 8000 is exposed and the container starts uvicorn directly (serving.main:app). The image intentionally does not train a model at build or start time: it expects models/current_model.joblib and models/scaler.joblib to already exist from a prior scripts/train.py run, since training is treated as a separate offline job whose promoted output is baked into or mounted onto the image before deployment. Build: docker build -t churn-api . — Run: docker run -p 8000:8000 churn-api. Section A.4 in the appendix shows this image built and running (churn-api:latest, 880 MB, 19 layers).

5. Data Pipeline and Retraining Strategy

New data arrives through the batch ingestion job described in Section 2. Retraining is not yet wired to a scheduler in this assignment, but the decision logic is implemented as a pure, unit-testable function ready to be dropped into a cron job or orchestrator DAG. Three signals are combined with OR logic, so any single one triggers a retraining recommendation:

Signal	Threshold	Severity
Staleness	≥ 14 days since the current model was trained	Medium
Performance regression	AUC on labeled feedback drops ≥ 0.03 vs. promotion-time AUC	High
Feature drift	Monitoring drift check flags a feature-mean shift > 20%	High

A retrain repeats the exact same pipeline as the initial run — pull the latest ingested data, rebuild features, train a new candidate, evaluate it against the current production model as the new baseline, and promote only if the guardrail passes — so every retrain is held to the same quality bar as the first training run. A newly promoted model would then roll out via a blue-green or canary deployment rather than an in-place swap, so traffic can be shifted gradually and reverted quickly if live metrics regress.

6. Monitoring Plan and Basic Alerts

Layer	Metrics	Alert threshold	Audience
Infra	Latency (avg, p95), error rate, volume	p95 > 300 ms or errors > 2% (5 min)	On-call / SRE
Data & feature	Rows/day, null rate, feature mean/std shift	Null rate > 5%, mean shift > 20%	ML engineer
Model & business	AUC on labeled feedback, positive rate	AUC below guardrail, rate > 2x range	Product / retention

Implemented lightweight check: a drift-check module runs a data-quality pass (missing required columns, null rates, out-of-range values) and a drift pass comparing each engineered feature's mean on a recent batch against the mean recorded at training time, flagging any feature whose mean has moved more than 20%. Run against the full ingested table, this produced 0 data-quality issues and 0 drift warnings, which is expected since the "recent batch" here is the same real data the model was just trained on. Both checks log their findings and write a JSON summary intended to feed downstream Grafana dashboards and Slack/PagerDuty alerts in a real deployment.

7. Incident Scenario

Scenario: the upstream billing system’s export job is updated so that MonthlyCharges starts arriving in cents instead of dollars, without notice to the ML team. Detection: the drift check, run against the next ingested batch, would flag two things — the data-quality range check would likely catch monthly_charges values far outside the expected range, and even if that were missed, the drift check would show monthly_charges and charges_per_tenure_month means shifted roughly 100x relative to the training reference, far past the 20% alert threshold. Tenure- and service-flag-derived features staying stable while only charge-derived features spike is itself a useful diagnostic clue that this is upstream data corruption rather than genuine behavioral drift. Response: immediately quarantine the corrupted batch so it cannot poison the labeled-feedback AUC calculation or the training table; do not retrain automatically even though the drift signal fires, since a human should confirm this is a data bug and not real-world drift first; patch ingestion validation to reject or auto-

convert malformed units going forward; and once the upstream fix is confirmed, re-ingest the corrected batch and resume normal monitoring.

Recovery time: detection under 1 hour (next ingestion + drift-check cycle), diagnosis 1–2 hours, fix and re-ingestion 2–4 hours — total incident duration under 8 hours, with the model continuing to serve on its last good weights throughout, so there is no prediction downtime.

8. Key Trade-offs, Limitations, and Future Work

- Simulated daily batches from a static file rather than a live dated feed — a deliberate scope trade-off for a two-week assignment; a real deployment would read from an actual dated warehouse table or event stream.
- A file-based registry.json instead of a full model-registry service (e.g. MLflow) — sufficient to demonstrate versioning and promotion, but would not scale to concurrent training jobs; a simple mean/std drift check instead of a statistical test such as a KS-test or Population Stability Index.
- Limitations: no authentication or rate-limiting on the API, no shadow or A/B deployment support for comparing candidate vs. baseline on live traffic, the retraining trigger is not yet wired to a scheduler, monitoring alerts are logged rather than sent to a real paging system, and the source data has no true timestamp so only structurally injected drift can currently be exercised, not genuine temporal drift.
- Future work: precision@k as a promotion metric, a real feature store for rolling-window features, shadow-mode evaluation before promotion, scheduler integration, and structured request logging for full traceability.

Overall, the project demonstrates a complete, if intentionally small-scale, production ML loop: a real dataset flows through batch ingestion into a shared feature module used identically by training and serving, a guardrail-gated promotion decision is made and logged to a versioned registry, the promoted model is served online with measured latency well inside budget, and a monitoring layer with a concrete incident scenario closes the loop back to retraining.

9. Architecture Diagram

Data flows from source → batch ingestion → an append-only training table. A single shared feature module is imported unchanged by both training and serving, eliminating skew by construction. The promoted model is tracked in a versioned registry, served via FastAPI, and packaged into the Docker container shown on the right of the serving layer; monitoring observes the serving and ingestion layers and feeds the retraining trigger that closes the loop back into training.

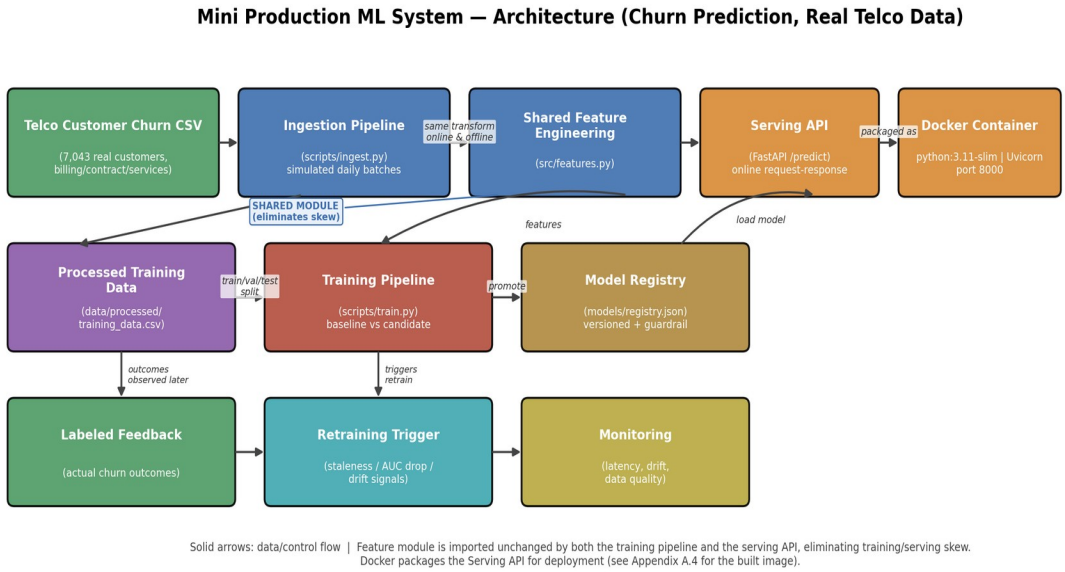
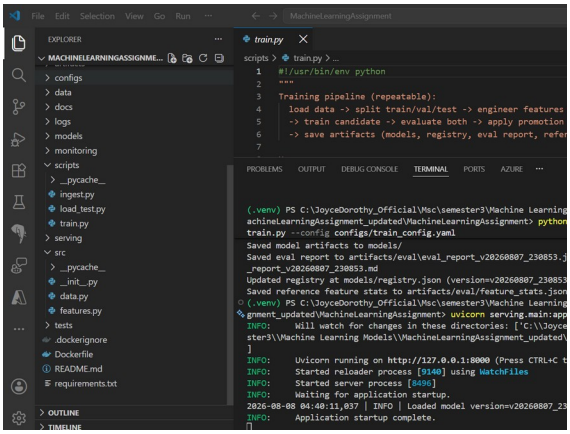


Figure 1. Data sources → ingestion → shared feature engineering → training → model registry → serving (FastAPI + Docker) → monitoring → retraining trigger.

Appendix: Testing and Run Evidence

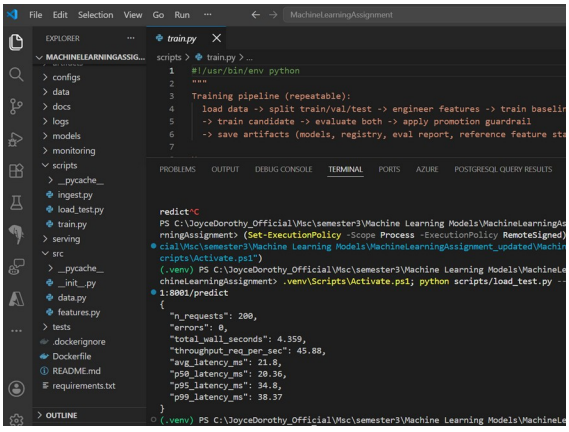
Screenshots from an actual local run of the full pipeline, confirming the training pipeline, serving API, load test, and Docker build all work end to end.

A.1 Training pipeline run



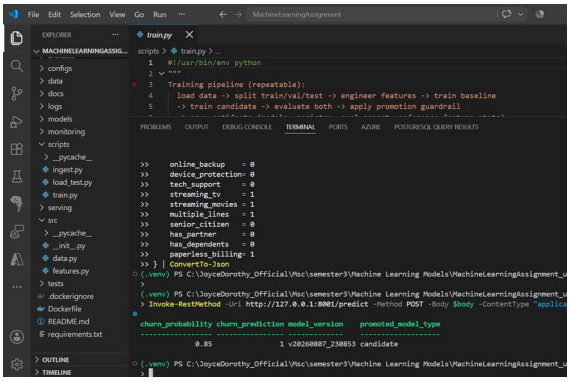
train.py ran end to end: artifacts, eval report, and registry.json saved; FastAPI then loaded the promoted model (v20260807_230853).

A.2 Load test



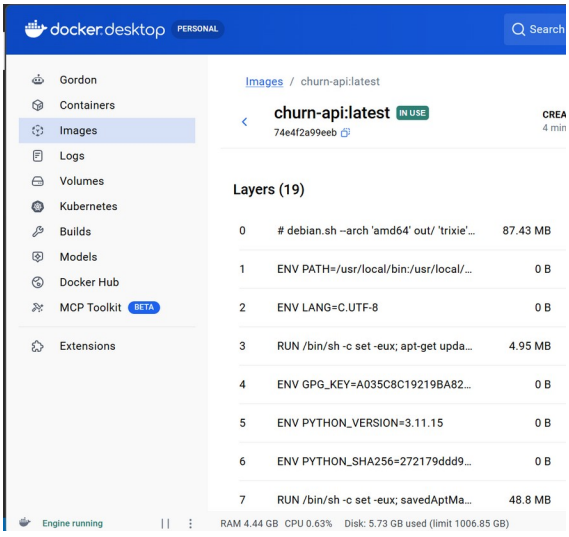
200 requests to /predict: 0 errors, 45.88 req/s, avg 21.8 ms, p95 34.8 ms, p99 38.37 ms.

A.3 Live /predict call



Invoke-RestMethod returns churn_probability = 0.85, churn_prediction = 1, model_version = v20260807_230853 — a high-risk customer correctly flagged.

A.4 Docker image build



churn-api:latest (880.34 MB, 19 layers, Python 3.11.15) built and running in Docker Desktop, shown "IN USE."